

Part 1 - data cleansing job

Before the code explanation, I owe you some organizational explanation. I organize the exercise around 4 parts that, despite composing the single pipeline, have different purposes. Putting them in a single exercise could lead to some implementation difficulties related to the problem understanding. This practice is also common in any real world problem solving. If the given problem is too big and difficult to apprehend, try to split it into different smaller but more approachable problems.

Exercise 1

Scala

Exercise 1 - data classification

The requirement can be solved with a simple *if-else* statement. The single challenge here is the JSON conversion to a struct so that we can access all the relevant attributes.

In the first job (JobExercise1) I used a minimalistic schema and from_json method:

```
val schemaForDataValidation = StructType(Array(
  StructField("user_id", LongType), StructField("visit_id", StringType)
))

val query = dataframe
  .select(
    $"key".cast("string"),
    functions.from_json($"value".cast("string"), schemaForDataValidation)
      .as("validation_data"),
    $"value".cast("string")
  )
  .withColumn("topic", functions.when(
    ($"validation_data.user_id" === 0 ||
    $"validation_data.visit_id".isNull), "invalid_data"
  ).otherwise("valid_data"))
```

The small schema avoids allocating memory for the fields that aren't useful in the data processing part. It's also a simplification factor because you reduce the evolution needs. It's more likely these 2 fields won't move that often than if we put other attributes and had to keep the whole schema up-to-date.

Exercise 2 - data deduplication

I'm still keeping the minimalistic schema but this time I have to add a new attribute, the `event_time`:

```
val schemaForDataValidation = StructType(Array(
  StructField("user_id", LongType), StructField("visit_id", StringType),
  StructField("event_time", TimestampType)
))
```

Why? As my deduplication mechanism I rely on Apache Spark's native capability provided with the `dropDuplicates` method. Under-the-hood it stores all the already seen events in an in-memory local storage that is written for fault-tolerance in a distributed data store. Typically the data store is an object store service.

Hence, this `dropDuplicates` method accumulates the data and our hardware is not unlimited. To mitigate the risk of Out-Of-Memory problems, Apache Spark provides also a cleaning mechanism for the too old seen events. This is the responsibility of the `event_time` attribute. Without this timestamp, Apache Spark wouldn't know how long it should keep each key in the memory storage.

It's too early in this module to deep dive into this. It's enough to know that the concept keeping data up to some time in memory is called watermark and in my code is defined as:

```
val query = dataframe
// ...
.withColumn("event_time", $"validation_data.event_time")
.withColumn("user_id", $"validation_data.user_id")
.withWatermark("event_time", "10 minutes")
.dropDuplicates("user_id", "event_time")
```

The watermark column must also be part of the deduplication columns. Otherwise Apache Spark won't be able to make a link between the processed records and their retention in the memory store.

Exercise 3 - data enrichment

Apparently, it's an easy part because it's a simple join between a static (aka *reference master*) dataset and our Apache Kafka streaming data.

```
val deviceReferenceDataset = sparkSession.read.format("jdbc")
  .options(JdbcReferenceDatasetConnectionOptions).load()

val enrichedRecordsToWrite = recordsToWrite.join(deviceReferenceDataset,
```

```

deviceReferenceDataset("device_type") ===
recordsToWrite("validation_data.technical.device.type")
).select(
  $"key", $"topic",
  functions.to_json(functions.struct($"value", $"full_name")).as("value")
)

```

However, as you already know from the Part 4 of the homework, there is maybe a data quality issue with the device type that may lead to the missed enrichment data. We're not going to address it at this moment and keep the issue for the next part.

Exercise 4 - data preparation

That's the final step. There are some changes in the code. First, I'm adding the data cleansing part as a map function calling the `toCleansedKeyWithValue` of the input JSON:

```

val recordsToWrite = dataframe
  .select(
    $"key".cast("string"),
    functions.from_json($"value".cast("string"), EventLog.Schema)
      .as("value")
  ).as[KeyWithValue]
  .map(eventLog => eventLog.toCleansedKeyWithValue)

```

The `toCleansedKeyWithValue` is defined at the case class representing input data. This approach has a readability advantage. The cleansed attributes don't leave the case class definition and we can easily find out the data preparation rules. Extracting the data preparation rules to a separate class is a valid approach, though, but in my opinion, less readable.

Since I clean the input data in the beginning of the processing, there is no more the join problem with the device. I can simply rely on the standardized `technical.device.type` attribute, even for the malformed events.

The next impacting part is the final output preparation. We have to include the `full_name` attribute from the reference dataset to the produced JSON. I'm doing it here with another map function that uses the input JSON if the device full name is not defined. For the sake of simplicity, I'm explicitly casting the low level `Row` object into more meaningful Scala classes:

```

val enrichedRecordsToWrite = recordsToWrite.join(deviceReferenceDataset,
  deviceReferenceDataset("device_type") ===
  recordsToWrite("value.technical.device.type"),
  "leftOuter"
).select($"key", $"value", $"topic", $"full_name")
.as[(String, CleansedEventLog, String, String)]
.map {
  case (key, value, topic, fullDeviceName) => {
    val oldTechnical = value.technical
    val valueToReturn = oldTechnical.device.map(oldDevice => {
      value.copy(technical = oldTechnical.copy(device =
        Some(oldDevice.copy(full_name = Some(fullDeviceName))))
    }).getOrElse(value)
    KafkaSinkParameters(key, valueToReturn, topic)
  }
}.select($"key", functions.to_json($"value").as("value"), $"topic")

```

The casting simplifies the processing code but also has a compute overhead because it requires objects initialization on the JVM side. Using or not using it is often a trade-off between a few milliseconds/seconds less in the processing but less readable code and potentially slower processing time but more readable and maintainable code base.

Python

Exercise 1 - data classification

Nothing changes here compared to the Scala version. The *if-else* statement also solves the data classification requirement:

```

schema_data_validation = StructType([
    StructField("user_id", LongType()), StructField("visit_id",
StringType())
])

query = input_data_stream.select(
    functions.col("key").cast("string"),
    functions.from_json(functions.col("value").cast("string"),
schema_data_validation).alias("validation_data"),
    functions.col("value").cast("string")
).withColumn("topic", functions.when(
    ((functions.col("value.user_id") == 0) |
(functions.col("value.visit_id").isNull())), "invalid_data")
    .otherwise("valid_data"))\
    .select("key", "value", "topic")

```

Exercise 2 - data deduplication

Again, the code relies on the same principle of dropDuplicates and an in-memory store for duplicates detection:

```
schema_data_validation = StructType([
    StructField("user_id", LongType()), StructField("visit_id",
StringType()),
    StructField("event_time", TimestampType())
])

query = input_data_stream.select(
    functions.col("key").cast("string"),
    functions.from_json(functions.col("value").cast("string"),
schema_data_validation).alias("validation_data"),
    functions.col("value").cast("string")
).withColumn("topic", functions.when(
    ((functions.col("value.user_id") == 0) |
(functions.col("value.visit_id").isNull())), "invalid_data")
    .otherwise("valid_data"))\
    .withColumn("event_time",
functions.col("validation_data.event_time"))\
    .withColumn("user_id", functions.col("validation_data.user_id"))\
    .withWatermark("event_time", "10 minutes")\
    .dropDuplicates(["user_id", "event_time"])\
    .select("key", "value", "topic")
```

Exercise 3 - data enrichment

Also here you won't find any difference with the Scala part. The API available for joining datasets is the same:

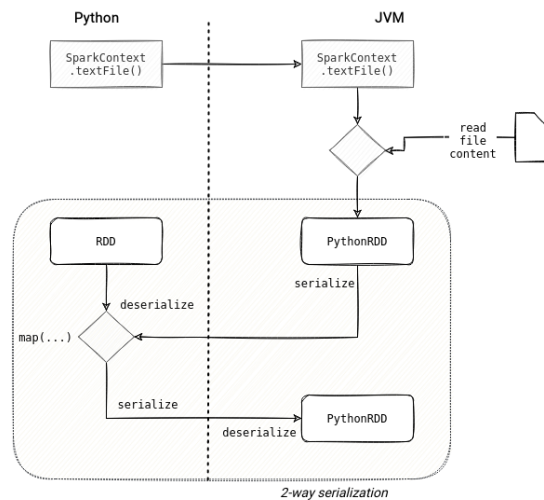
```
enriched_dataset_to_write = query.join(device_reference_dataset,

device_reference_dataset["device_type"] ==
query["validation_data.technical.device.type"], "left_outer").select(
    functions.col("key"), functions.col("topic"),
    functions.to_json(functions.struct("value",
"full_name")).alias("value")
)
```

Exercise 4 - data preparation

Only this final step is different between Scala and Python versions. PySpark doesn't provide the same mapping capacity as the Scala Spark API. The map function is only available for the RDD abstraction and using it in streaming processing which by definition is more time-sensitive than the batch one is problematic because it requires a 2-way serialization. Let's take an example.

We wrote some code to read text files and apply a custom mapping on top of each line. When we run the job, the following will happen:



PySpark and RDD 2-way serialization - simplified

As you can see in the picture, there is a duplicated serialization effort per line, since the map applies individually for each row. As a consequence, the job can suffer serious performance issues.

To mitigate them and still allow users to apply custom mapping, PySpark comes with some Python-native solutions. Let's use the first of them to implement the data cleansing part:

```

@pandas_udf(MapType(StringType(), StringType()))
def clean_value(values_to_clean: pandas.Series) -> pandas.Series:
    values_to_clean_as_dict = values_to_clean.to_dict()
    values_to_return = {}
    for index, json_to_clean in values_to_clean_as_dict.items():
        json_value = json.loads(json_to_clean)
        json_value['source'] = Cleanser.cleanser_source(json_value)
        json_value['technical'] =
        Cleanser.cleanser_technical(json_value['technical'])
        topic = 'invalid_data' if json_value['user_id'] == 0 or not
        json_value['visit_id'] else 'valid_data'
        values_to_return[index] = {'payload': json.dumps(json_value),
                                   'device':
        json_value['technical']['device']['type'],
                                   'topic': topic}
    return pandas.Series(values_to_return)

query = input_data_stream.select(
    functions.col("key").cast("string"),
    functions.from_json(functions.col("value").cast("string"),
    schema_deduplication).alias("deduplication_columns"),
    functions.col("value").cast("string")) \
    .withColumn("event_time",

```

```

functions.col("deduplication_columns.event_time")) \
    .withColumn("user_id",
functions.col("deduplication_columns.user_id")) \
    .withWatermark("event_time", "10 minutes") \
    .dropDuplicates(["user_id", "event_time"]) \
    .withColumn("cleansed_value",
mappers.clean_value(functions.col("value")))\
    .selectExpr("key", "cleansed_value.payload AS value",
"cleansed_value.device", "cleansed_value.topic")

```

The solution relies on a Vectorized User-Defined Function that unlike a normal User-Defined Function, works on a bunch of rows at once. It's a significant performance gain over the classical map function because it reduces the serialization cost a lot.

The next challenge for this part was adding the `full_name` attribute to the device part. Again, I have to do something PySpark-specific:

```

def enrich_output_with_full_device_name(input: pandas.DataFrame) ->
pandas.DataFrame:
    def combine_with_full_name(value_json: str, device_full_name: str) ->
str:
        json_dict = json.loads(value_json)
        json_dict['technical']['device']['full_name'] = device_full_name
        return json.dumps(json_dict)

    for rows in input:
        rows['value'] = rows.apply(lambda single_row:
combine_with_full_name(
            single_row['value'], single_row['full_name']), axis=1
        )
        yield rows

data_to_write =
enriched_dataset_to_write.mapInPandas(enrich_output_with_full_device_nam
e, enriched_dataset_to_write.schema)\
    .select(
        functions.col("key"),
        functions.col("value"),
        functions.col("topic")
    ).selectExpr("key", "cleansed_value.payload AS value",
"cleansed_value.device", "cleansed_value.topic")

```

This time I'm using a dedicated PySpark mapping function called `mapInPandas`. Why this instead of the UDF? Semantically speaking, the integration of the `full_name` attribute is more an operation on all the columns, so mapping seems more appropriate than a typically column-based UDF.

When it comes to the `mapInPandas`, it shares the logic with the Vectorized UDF. Hence, it takes a bunch of records at once to produce the output. The size of the batch is defined in `spark.sql.execution.arrow.maxRecordsPerBatch` and it's set to 10 000 by default.

Best,
Bartosz.